

Digit Object Detection in Street View Images Using DETR

Project Report

Subject

[S25] Deep Learning with Python



Data ScienceTech Institute

Instructor

Benoit Mialet

Submitted by

Shikhar Saini, Kevin Blinker, Lyna Mouhoubi, Youssef Mhiri

DSTI School of Engineering

30th September 2026

Table of contents

Abstract	2
1. Introduction	2
1.1 Problem Statement	2
2. Data Understanding	2
2.1 Dataset	2
2.2 Short EDA	3
Image characteristics	3
Number of digits	3
Class distribution	3
Bounding boxes	3
3. Preprocessing + Training Setup	3
3.1 Annotation Conversion	3
SVHN annotations are provided as:	3
For DETR:	3
3.2 Image Processing	4
3.3 Repository Structure and Model Locations	4
6. Training	4
4.1 Experimental context	4
Stage 1 – Controlled model comparison	4
Stage 2 – Focused DETR experiment	5
7. Results	5
8. Discussion	7
6.1 Model Comparison	7
6.2 Conclusion	7
9. Extra Work : Azure ML Experience & Limitations	8
10. Reference	8
11. Annexure	9
Annexure I - Resultant images from Baseline model for Detr (1 Epoch)	9
Annexure II - Resultant images from Main model for Detr	9
Annexure III - Tests on Azure MLFoundary	10
III.1 Azure ML Setup	10
III.2 Environment and Dataset Smoke Tests	10
III.3 Local Pipeline Validation	10
III.4 Azure Compute Selection and GPU Limitation	10
III.5 ML foundry Jobs	11
III.5.1 Logs for Final Azure Test	11
Annexure IV - Git Repository File Structure	12

Abstract

This project investigates deep learning approaches for multi-object digit detection on the SVHN dataset. YOLOv8, SSDLite, Faster R-CNN, and DETR with ResNet-50 were evaluated using mAP, mAP@50, and mAP@75. DETR achieved an mAP of **0.421** in the initial comparison and was subsequently trained on the full dataset for 10 epochs, achieving **0.423 mAP, 0.778 mAP@50, and 0.326 mAP@75**. The project also used extensive smoke testing and Azure Machine Learning to support model training, evaluation, and experiment tracking.

1. Introduction

1.1 Problem Statement

- **Task:** multi-object detection of handwritten/printed digits in street-view images.
- **Dataset:** SVHN (full_numbers configuration).
- **Objective:** detect every digit in an image and assign the correct digit class (0–9) to each bounding box.
- **Output:** bounding boxes + digit class for each detected digit.
- **Why interesting:**
 - Multiple digits can appear in one image.
 - Images vary substantially in size, background, lighting, and digit scale.
 - The task requires both localization and classification.
 - DETR provides an end-to-end transformer-based object-detection approach that can be fine-tuned from a pretrained model.

2. Data Understanding

2.1 Dataset

SVHN — full_numbers

Original Data :-

Split	Images
Train	33,402
Test	13,068
Extra	202,353

Our working Split :-

Split	Images	Purpose
Training	30061 / 3500	Model optimization
Validation	3341 / 750	Model selection / hyperparameter decisions
Test	13,068	Final evaluation

- Each sample contains:
 - an image
 - digit annotations, consisting of bounding boxes and corresponding labels (0–9).
- Bounding boxes are originally represented as [x, y, width, height].

- Images have variable dimensions and can contain multiple digits.

For this project, the original training split is divided into 90% training / 10% validation, using seed 42. The official test set remains held out for final evaluation.

2.2 Short EDA

Only the most important findings.

Image characteristics

- Training image dimensions are highly variable.
- Mean training image size approximately **128 × 57 pixels**.
- Long-tailed image dimensions exist.

Number of digits

- Mean: approximately **2.19 digits/image** in training.
- Median: **2 digits/image**.
- Range: 1–6 digits.

Class distribution

- Digit **1** is the most frequent class.
- Digit **9** is the least frequent.
- The most/least frequent class ratio is approximately **3:1**.
- No class weighting is applied initially; per-class performance will instead be monitored.

Bounding boxes

- 73,257 annotated digit boxes in the training split.
- Bounding-box dimensions vary considerably.
- This creates variation in object scale that is relevant to detector performance.

3. Preprocessing + Training Setup

3.1 Annotation Conversion

SVHN annotations are provided as:

[x, y, width, height]

For DETR:

- annotations are converted into the format expected by `DetrImageProcessor`;
- bounding boxes are represented internally in normalized center-based coordinates;
- images are resized while preserving aspect ratio;
- normalization is performed using the pretrained processor;

- images in a batch are padded to a common size.

3.2 Image Processing

Baseline preprocessing will use the pretrained DETR processor. We should later document the exact processor configuration used in the Azure baseline.

Potential controlled experiment:

- **Baseline:** default DETR preprocessing
- **Experiment:** reduced input resolution

3.3 Repository Structure and Model Locations

All implemented models and experiments are available in the project repository. The main model implementations can be found in the following locations:

- **DETR Baseline:** [DETR_BaselineModel/](#)
- **DETR Final Model:** [DETR_FinalModel/](#)
- **YOLOv8 + SSDLite:** [Notebooks/03_yolov8_ssd-lite.ipynb](#)
- **Faster R-CNN ResNet-50:** [Notebooks/04_faster_rcnn.ipynb](#)

Additional project notebooks are also provided:

4. **EDA:** [Notebooks/01_eda_svhn.ipynb](#)
5. **Smoke Test:** [Notebooks/02_smoke_test.ipynb](#)

6. Training

4.1 Experimental context

Stage 1 – Controlled model comparison

- The first stage evaluated several object-detection architectures on the **SVHN full_numbers** dataset. To make the comparison more consistent, the experiments used:
 - **3,500 training images**
 - **750 validation images**
 - **5 training epochs**
 - **Batch size: 4**
- The main purpose was to compare the performance of:
 - YOLOv8
 - SSDLite
 - Faster R-CNN ResNet-50
 - DETR ResNet-50
- **Learning rate and weight decay were not fixed across models.**
 - These hyperparameters were adjusted for each particular architecture.

- The reason was that different detection architectures can respond differently to optimization settings.
 - The experiments therefore aimed to identify a suitable learning rate and weight decay configuration for **good performance for each individual model**.
- This stage was conducted under limited computational resources, so using a reduced dataset and fixed number of epochs allowed multiple architectures to be experimentally evaluated within a practical training budget.

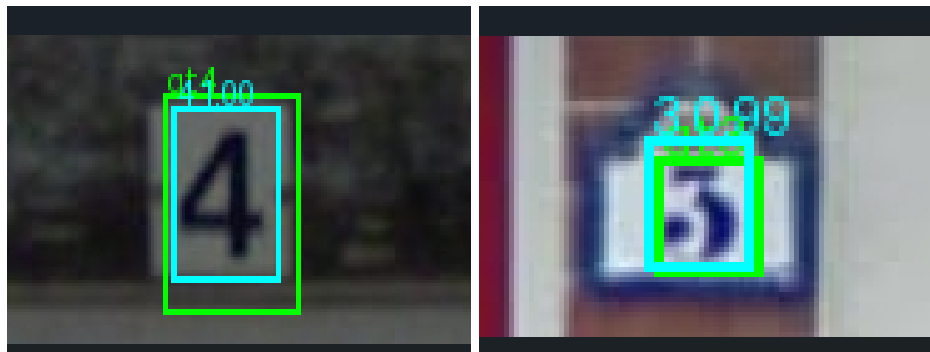
Stage 2 – Focused DETR experiment

- The Stage-1 comparison identified DETR ResNet-50 as the model selected for further experimentation based on its Stage-1 result:
 - mAP: 0.421
 - mAP50: 0.744
 - mAP75: 0.211
- DETR was subsequently trained using a substantially larger dataset:
 - Training images: 30,061
 - Validation images: 3,341
 - Epochs: 10
 - Batch size: 4
 - Learning rate: 0.0001
 - Weight decay: 0.0001
- The purpose of this second stage was to investigate whether increasing the available training data and training duration could further improve DETR performance.
- This experiment required substantially more computational resources, so the full-data experiment was performed only for the selected model.

7. Results

	Stage - I				Stage - II	
Model	YOLOv8	SSDLite	Faster R-CNN ResNet-50	DETR ResNet-50	DETR ResNet-50 (Baseline)	DETR ResNet-50 (Final Model)
GPU	NVIDIA Tesla T4	NVIDIA Tesla T4	NVIDIA T4	NVIDIA GeForce RTX 3050 Ti	NVIDIA GeForce RTX 3050 Ti	NVIDIA GeForce RTX 4070
Training Time	31.82 min	50.28 min	55.68 min	8.2 hrs	15.1 hrs	10 hrs
Training Split	3500	3500	3500	3,500	30061	30061
Validation Split	750	750	750	750	3341	3341
Epochs	10	10	5	5	1	10
Batch Size	16	8	4	4	4	4
Learning Rate	0.000714	0.001	0.002	0.0001	0.00001	0.0001
Weight Decay	0.0005	0.01	0.005	0.0001	0.0001	0.0001
mAP	0.2942	0.2566	0.3153	0.421	0.0086	0.423
mAP50	0.6989	0.6414	0.7408	0.744	0.0197	0.778
mAP75	0.1786	0.1382	0.1943	0.211	0.0056	0.326

Below are the results from best model : *DETR ResNet-50 (Final Model)*



8. Discussion

6.1 Model Comparison

- YOLOv8, SSDLite, Faster R-CNN, and Stage-1 DETR were part of the model-comparison experiment using a limited dataset.
- The learning rate and weight decay vary because these were treated as model-specific hyperparameters and tuned to obtain suitable performance for each architecture.
- DETR Stage 1 achieved:
 - mAP: 0.421
 - mAP50: 0.744
 - mAP75: 0.211
- This Stage-1 result motivated the decision to investigate DETR further using the full available training and validation splits.
- The original DETR baseline is retained in the table as a reference point:
 - 1 epoch
 - 30,061 training images
 - 3,341 validation images
 - learning rate 1e-5
 - mAP 0.0086
- The final DETR experiment changed the training setup to 10 epochs and learning rate 1e-4, while using the same full dataset scale.
- The final experiment achieved:
 - mAP: 0.423
 - mAP50: 0.778
 - mAP75: 0.326

6.2 Conclusion

The results should not be interpreted as a perfectly controlled architecture benchmark, since the models were trained with different dataset splits, numbers of epochs, batch sizes, learning rates, GPUs, and computational budgets.

The model-specific hyperparameter experimentation allowed the team to identify suitable learning-rate and weight-decay configurations for the individual architectures before selecting DETR for a more extensive experiment.

The extensive smoke testing performed by the team helped validate data loading, preprocessing, model initialization, training, evaluation, and checkpointing before the final experiments.

Overall, the experiments demonstrate that model architecture, training configuration, dataset scale, and available computational resources all played an important role in the observed results, while providing practical experience with several modern object-detection approaches for SVHN.

9. Extra Work : Azure ML Experience & Limitations

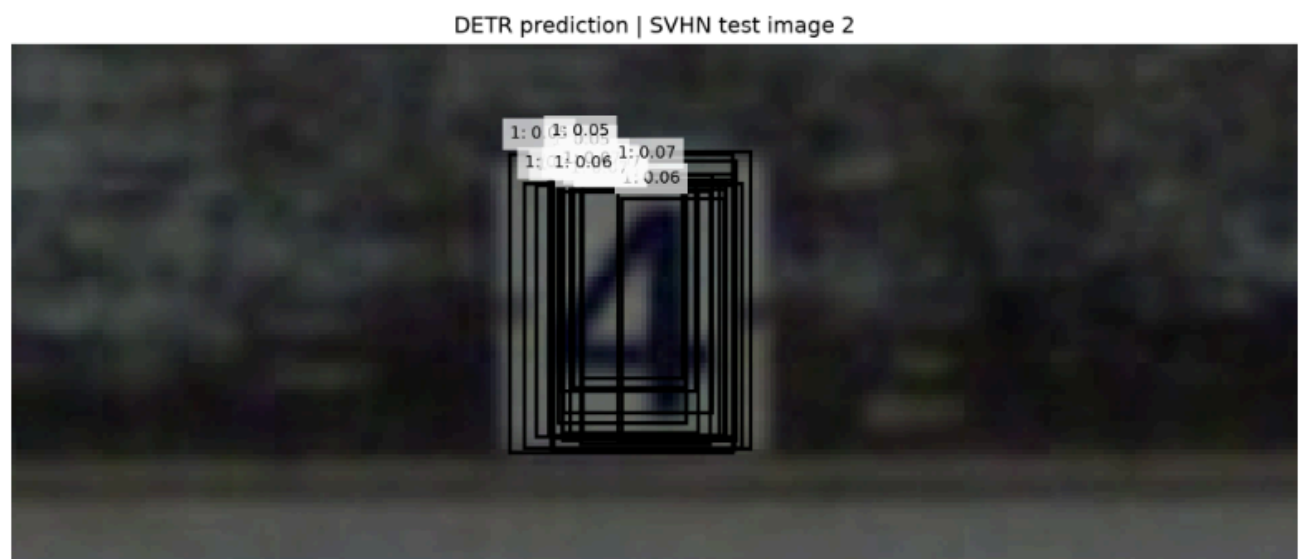
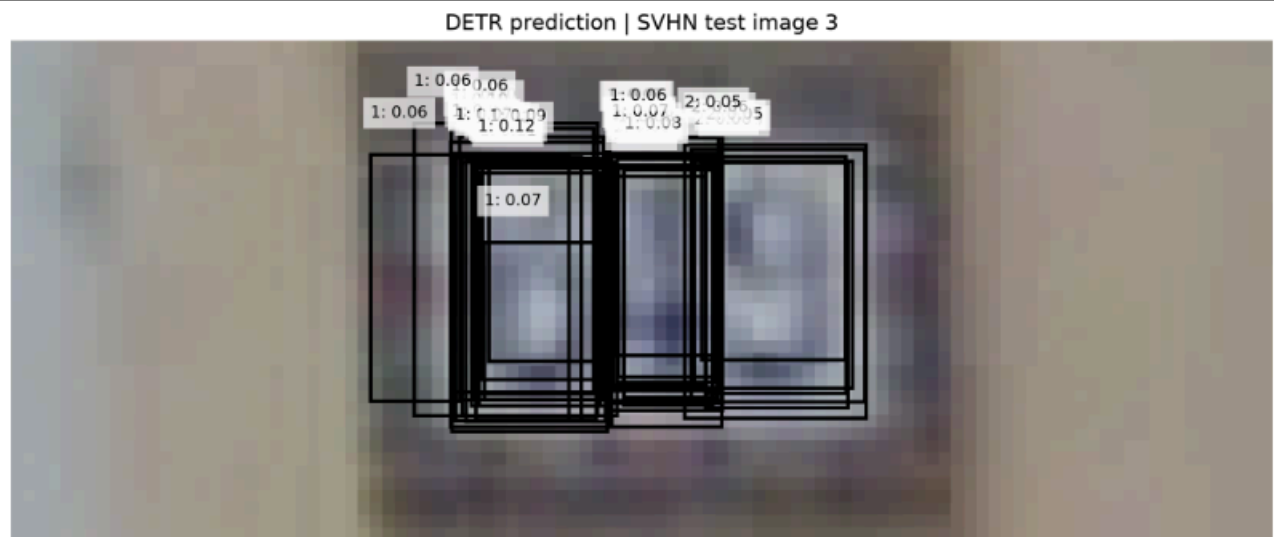
- We successfully **deployed and ran the DETR deep-learning object-detection algorithm on Azure Machine Learning**, using the SVHN dataset and a managed Azure ML training environment.
- Azure ML was used for **dataset access, environment management, training execution, MLflow tracking, and checkpoint management**.
- The Azure experiment successfully demonstrated that our complete deep-learning training pipeline could run remotely in the cloud.
- A major limitation was the **lack of GPU quota** under the Azure for Students subscription, so the Azure training experiment was performed using CPU compute.
- Due to the slower CPU training, longer experiments were conducted using the available local/team GPU resources.
- To know more about our work with Azure refer to [Annexure - III](#)

10. Reference

- ❖ Github link : <https://github.com/sr1009/huggingface-object-detection>

11. Annexure

Annexure I - Resultant images from Baseline model for Detr (1 Epoch)



Annexure III - Tests on Azure MLFoundary

III.1 Azure ML Setup

To extend the project from local experimentation to a cloud-based workflow, an **Azure Machine Learning** workspace was created using an Azure for Students subscription. A dedicated resource group, [DL_Grp_project_ObjDec](#), and workspace, [AML_SVHN_ObjDetection](#), were created in the **Sweden Central** region. Team members were given the **AzureML Data Scientist** role to enable collaborative access to the workspace.

Azure ML was used to manage the training environment, compute resources, experiment execution, MLflow tracking, and model artifacts. The project was later integrated into the main repository structure so that the same codebase could be used for both local and Azure ML experiments.

III.2 Environment and Dataset Smoke Tests

Before running computationally expensive experiments, several smoke tests were performed.

An **environment test** verified the availability of the main project dependencies, including PyTorch, Transformers, Hugging Face Datasets, and MLflow. It also checked CUDA availability and confirmed that the [facebook/detr-resnet-50](#) model could be loaded and adapted for the project's ten digit classes (0–9).

A separate **dataset access test** loaded the public SVHN dataset from Hugging Face using the [full_numbers](#) configuration. The test verified that the dataset was accessible and that images, digit labels, and bounding-box annotations could be read successfully.

A basic **Azure ML smoke test** was also used to confirm that Python jobs could start correctly within the Azure ML environment and access their working directory.

III.3 Local Pipeline Validation

Before submitting the complete training job to Azure, the training pipeline was tested locally using small subsets of the dataset. These tests validated the complete training flow, including:

- Dataset loading and train/validation splitting
- Image preprocessing and bounding-box conversion
- DETR model initialization
- Batch collation
- Forward and backward passes
- Loss calculation
- Validation and detection metrics
- Checkpoint creation

These preliminary tests were important because they allowed implementation errors to be identified without consuming significant cloud resources.

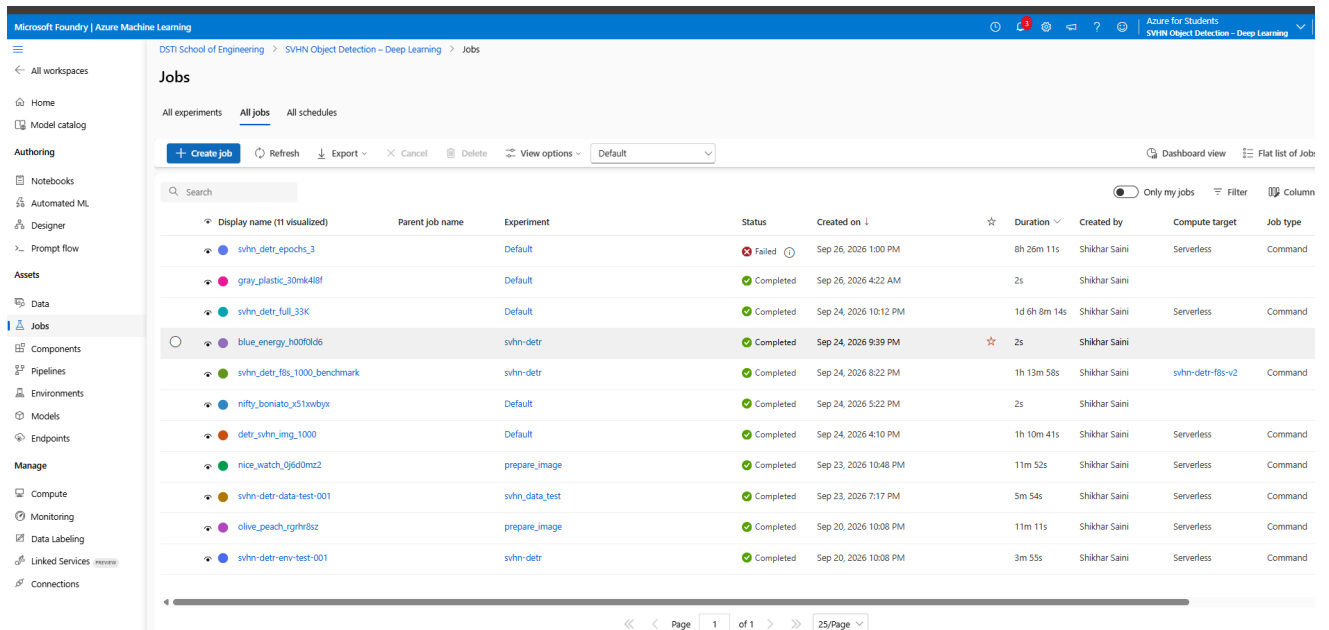
III.4 Azure Compute Selection and GPU Limitation

Different Azure CPU configurations were tested to identify a suitable option for the project. The **D3s_v2** configuration was selected for the main Azure experiment because it provided a reasonable balance between computational performance and cost.

GPU-based Azure training was also investigated. However, the Azure for Students subscription could not obtain the required GPU quota. Consequently, longer GPU experiments were performed using available local/team GPU resources, while Azure was primarily used to validate the cloud-based training workflow.

The complete CPU training run required approximately **30 hours**, demonstrating the computational cost of running DETR on CPU infrastructure.

III.5 ML foundry Jobs



Display name (11 visualized)	Parent job name	Experiment	Status	Created on	Duration	Created by	Compute target	Job type
svhn_detr_epochs_3		Default	Failed	Sep 26, 2026 1:00 PM	8h 26m 11s	Shikhar Saini	Serverless	Command
gray_plastic_30mk48f		Default	Completed	Sep 26, 2026 4:22 AM	2s	Shikhar Saini		
svhn_detr_full_33K		Default	Completed	Sep 24, 2026 10:12 PM	1d 0h 8m 14s	Shikhar Saini	Serverless	Command
blue_energy_h00f0d6	svhn_detr		Completed	Sep 24, 2026 9:39 PM	2s	Shikhar Saini		
svhn_detr_fbs_1000_benchmark	svhn_detr		Completed	Sep 24, 2026 8:22 PM	1h 13m 58s	Shikhar Saini	svhn-detr-fbs-v2	Command
nifty_boniato_x51wbxye	Default		Completed	Sep 24, 2026 5:22 PM	2s	Shikhar Saini		
detr_svhn_img_1000	Default		Completed	Sep 24, 2026 4:10 PM	1h 10m 41s	Shikhar Saini	Serverless	Command
nice_watch_0j6d0mc2	prepare_image		Completed	Sep 23, 2026 10:48 PM	11m 52s	Shikhar Saini	Serverless	Command
svhn_detr-data-test-001	svhn_data_test		Completed	Sep 23, 2026 7:17 PM	5m 54s	Shikhar Saini	Serverless	Command
olive_peach_rghr8sz	prepare_image		Completed	Sep 20, 2026 10:08 PM	11m 11s	Shikhar Saini	Serverless	Command
svhn_detr-env-test-001	svhn_detr		Completed	Sep 20, 2026 10:08 PM	3m 55s	Shikhar Saini	Serverless	Command

III.5.1 Logs for Final Azure Test

Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN to enable higher rate limits and faster downloads.

Experiment: svhn_detr_baseline

Device: cpu

View run svhn_detr_full_33K at:

https://swedencentral.api.azureml.ms/mlflow/v2.0/subscriptions/5d142fcb-099d-4666-a58b-f0626635520a/resourceGroups/dl_grp_project_objdec/providers/Microsoft.MachineLearningServices/workspaces/AML_SVHN_ObjDetection/#!/experiments/4e2ba3fd-850e-44fc-9cdf-271819dca510/runs/svhn_detr_full_33K

View experiment at:

https://swedencentral.api.azureml.ms/mlflow/v2.0/subscriptions/5d142fcb-099d-4666-a58b-f0626635520a/resourceGroups/dl_grp_project_objdec/providers/Microsoft.MachineLearningServices/workspaces/AML_SVHN_ObjDetection/#!/experiments/4e2ba3fd-850e-44fc-9cdf-271819dca510

Generating train split: 0 examples [00:00, ? examples/s]

Generating train split: 5600 examples [00:00, 49759.24 examples/s]

Generating train split: 13600 examples [00:00, 66388.00 examples/s]

Generating train split: 21000 examples [00:00, 67047.70 examples/s]

Generating train split: 28400 examples [00:00, 67410.44 examples/s]

Generating train split: 33402 examples [00:00, 66274.73 examples/s]

[transformers] You passed `num\_labels=10` which is incompatible to the `id2label` map of length `91`.

Train images: 30061

Validation images: 3341

Train batches: 7516

Validation batches: 836

Loading weights: 0%| | 0/530 [00:00<?, ?it/s]

Loading weights: 87%|██████████ | 463/530 [00:00<00:00, 4623.26it/s]

Loading weights: 100%|██████████| 530/530 [00:00<00:00, 4714.28it/s]

[transformers] [1mDetrForObjectDetection LOAD REPORT[0m from: facebook/detr-resnet-50

Key	Status
-----	--------

model.backbone.model.layer3.0.downsample.1.num_batches_tracked	UNEXPECTED
--	------------

model.backbone.model.layer1.0.downsample.1.num_batches_tracked	UNEXPECTED
--	------------

model.backbone.model.layer2.0.downsample.1.num_batches_tracked	UNEXPECTED
--	------------

model.backbone.model.layer4.0.downsample.1.num_batches_tracked	UNEXPECTED
--	------------

class_labels_classifier.weight	MISMATCH	Reinit due to size mismatch - ckpt: torch.Size([92, 256]) vs model:torch.Size([11, 256])
--------------------------------	----------	--

class_labels_classifier.bias	MISMATCH	Reinit due to size mismatch - ckpt: torch.Size([92]) vs model:torch.Size([11])
------------------------------	----------	--

Notes:

- UNEXPECTED: can be ignored when loading from different task/architecture; not ok if you expect identical arch.

- MISMATCH: ckpt weights were loaded, but they did not match the original empty weight shapes.

Epoch 1/1 | train_loss=1.7038 | val_loss=1.5355 | val_mAP=0.0040 | val_AP50=0.0099 | val_AP75=0.0028 | BEST

Azure ML detected: checkpoint will be persisted through ./outputs instead of MLflow artifact logging.

Checkpoint saved to: outputs/svhn_detr_baseline/best_checkpoint.pt

Experiment completed.

Annexure IV - Git Repository File Structure

